VARDHAMAN
COLLEGE OF ENGINEERING

A Course End Project Report on

Text Encryption and Decryption

A9513- Python Programming Laboratory

Submitted by

Nomula Nishitha
25881A05A2

Addanki Vijay
Kumar
25881A05C9

Course Facilitator

Ms P.S. Madhavi
Assistant Professor

Department of Computer Science and Engineering

Vardhaman College of Engineering, Hyderabad

VARDHAMAN
COLLEGE OF ENGINEERING

Autonomous Institute, Affiliated to **JNTUH,**
Approved by **AICTE,** Accredited by **NAAC**
with **A++** Grade Kacharam, Shamshabad,
Hyderabad- 501 218, Telangana, India.

Vibrant Vardhaman
25 Years of

DEPARTMENT OF COMPUTER SCIENCE AND ENGINEERING

This is to certify that the Course End Project titled **"Text Encryption and Decryption"** is carried out by **Nomula Nishitha** and **Addanki Vijay Kumar** with Roll Numbers **25881A05A2 and 25881A05C9** towards **A9513 - Python Programming Laboratory** course in partial fulfilment of the requirements for the award of degree of **Bachelor of Technology** in **Computer Science and Engineering** during the Academic year 2026-27.

Signature of the Course Faculty Ms P. S. Madhavi
Assistant Professor, CSE

Signature of the HOD
Dr .Y. Vijayalata
Professor, Head of the Department & Dean(Student Mentoring)
Department Computer Science and Engineering,
Vardhaman College of Engineering, Hyderabad.

Contents

1 Introduction 3
2 Literature Review 3
3 Objectives 3
4 Methodology 3
4.1 Architectural Diagram 3
4.2 Hardware and Software Requirements 3
4.3 Working Principle 4
4.4 Software Implementation 4
5 Results and Discussion 4
6 Mapping POs , PSOs and SDGs 5
6.1 Program Outcomes (POs) Mapping 5
6.2 Sustainable Development Goals (SDGs) Mapping 5
6.3 Program Specific Outcomes (PSOs) Mapping 5
7 Conclusion 5
Bibliography 6

Abstract

This project presents CipherLab, a Python application for demonstrating text encryption and decryption through a graphical user interface built with Tkinter. The application brings together six classical methods: Caesar, Atbash, Vigenère, Affine, Rail Fence, and Columnar Transposition. Users can enter a message, select a method and operation, provide the required key, and view the resulting text. The interface validates key formats, such as whole numbers for Caesar and Rail Fence, a pair of values for Affine, and alphabetic keywords for Vigenère and Columnar Transposition. Atbash does not require a key.

The program is organized into three sections. The Encrypt / Decrypt section processes a message with the selected method and provides a step-by-step explanation of how the transformation works. The Compare 6 Methods section encrypts the same input using a demonstration key for each method and displays the outputs and their lengths in a table. This lets users observe how different algorithms transform identical text. The Analyze / Crack Demo section generates possible decryptions. It tries all 26 Caesar shifts, applies the Atbash transformation, tests the 312 valid Affine key combinations, and tries a range of Rail Fence rail counts. For Vigenère and Columnar Transposition, it decrypts using candidate keywords supplied by the user. These are possible results for inspection and are not guaranteed to identify the original message.

The project demonstrates Python functions, loops, conditional statements, string and character handling, input validation, modular arithmetic, and event-driven GUI programming. It also introduces the differences between substitution methods, which replace letters in place, and transposition methods, which rearrange character positions. Because the transposition methods rearrange the input, spaces and punctuation may move along with the other characters.

CipherLab is intended as an educational tool for exploring classical encryption methods and comparing their outputs. These methods have known weaknesses and are not suitable for protecting confidential information.

Keywords: Python, Tkinter, text encryption, text decryption, Caesar cipher, Atbash cipher, Vigenère cipher, Affine cipher, Rail Fence cipher, Columnar Transposition, cipher comparison, cryptanalysis.

1 Introduction

Text encryption and decryption are processes for transforming readable information and then recovering it. Encryption changes an original message into a different form according to a method and a key. Decryption applies the corresponding reverse process to recover the original message. Learning how these transformations work provides an introduction to cryptography and to the ways computer programs handle text.

This project, **CipherLab**, is a Python application with a graphical user interface developed using Tkinter. It includes six classical methods: Caesar, Atbash, Vigenère, Affine, Rail Fence, and Columnar Transposition. The methods demonstrate different approaches to changing a message. Caesar, Atbash, Vigenère, and Affine substitute letters according to a rule, while Rail Fence and Columnar Transposition rearrange the positions of characters. Each method has its own key requirements. For example, Caesar uses a whole-number shift, Vigenère uses a keyword, Affine uses two values, and the transposition methods use a rail count or keyword.

CipherLab is organized into three sections. In **Encrypt / Decrypt**, the user enters text, selects a method and operation, provides a key, and views the result. The program also displays an explanation of the steps used by the selected method. In **Compare 6 Methods**, the same message is encrypted using each method's demonstration key, and the resulting text and lengths are shown in a table. This allows users to observe how the selected method affects the output. In **Analyze / Crack Demo**, the program generates possible decrypted results. It tries all shifts for Caesar, the reverse-alphabet mapping for Atbash, valid key combinations for Affine, and a range of rail counts for Rail Fence. For Vigenère and Columnar Transposition, it tests keywords entered by the user.

The interface validates key formats and displays error messages when the input does not match the selected method. The project uses Python functions, loops, conditional statements, strings, character conversion, modular arithmetic, and event-driven programming with Tkinter. It provides a practical way to connect these programming concepts with introductory cryptography.

All six methods in the application are classical ciphers intended for learning and demonstration. The candidate results in the analysis section are possibilities for the user to examine and may not reveal the original message. These methods should not be used to protect confidential information.

2 Literature Review

Classical ciphers introduce the basic ideas of encryption, decryption, keys, and transformations of text. They are commonly grouped into substitution methods, which replace letters according to a rule, and transposition methods, which rearrange character positions. Bruen, Forcinito, and McQuillan (2021) discuss historical ciphers, including Caesar and Vigenère, and the cryptanalysis techniques that can be used to study them. These methods are useful for teaching because their operations can be demonstrated with simple examples and calculations.

The Caesar cipher shifts each letter by a fixed number of places in the alphabet. Atbash uses a fixed reverse-alphabet mapping, while the Affine cipher calculates a substitution using two numeric values and modular arithmetic. The Affine method is reversible only when its first key value has a multiplicative inverse modulo 26. The Vigenère cipher uses a repeating keyword to vary the shift applied to each letter. Although it produces different shifts across a message, repeated-key patterns can still be analyzed. Research on Vigenère cryptanalysis describes methods such as estimating keyword length and examining repeated sequences to help recover possible keys (Al-Sabaawi, 2020; Hananto et al., 2019).

Rail Fence and Columnar Transposition are transposition methods. Rail Fence writes characters in a zigzag pattern across a chosen number of rows, then reads each row in sequence. Columnar Transposition writes characters across rows and reads the columns according to the alphabetical order of a keyword. Unlike substitution methods, these techniques change character positions rather than replace each letter in its original position. The CipherLab application includes both methods to demonstrate this difference.

CipherLab also provides a graphical interface, step-by-step explanations, a comparison table, and an analysis feature that generates possible decryptions. The analysis tries fixed or limited key spaces for some methods and user-supplied keyword candidates for Vigenère and Columnar Transposition. These candidates help illustrate how cipher analysis works, but they do not guarantee recovery of the original message. Tkinter supports the application's event-driven graphical interface (Python Software Foundation, "Tkinter Documentation").

The six methods are included as educational examples, not modern security systems. Their limited key spaces and recognizable patterns make them unsuitable for protecting confidential information. The project uses them to compare substitution and transposition techniques and to practise Python programming concepts.

9:43 PM

3 Objectives

• To design and develop a Python application that implements six text encryption and decryption methods.

The project aims to create one application that demonstrates six classical ciphers: Caesar, Atbash, Vigenère, Affine, Rail Fence, and Columnar Transposition. Users can select a method, choose encryption or decryption, and enter its required key. The program should apply the selected transformation and display the result. Supporting different methods allows users to observe how substitution ciphers change letters and how transposition ciphers rearrange characters.

• To create a graphical user interface that is clear and easy to use.

The application uses Tkinter to provide separate sections for processing a message, comparing methods, and analyzing possible decryptions. Users can enter text, choose a cipher and operation, provide a key, and view the output in the application window. The interface also displays step-by-step explanations to help users understand how each method processes the message.

• To provide a comparison of the six methods using the same input.

The comparison feature encrypts the same message with each cipher using a demonstration key. It displays the method name, key, output, and output length in a table. This helps users compare the transformations produced by the different methods. The comparison is intended to show how the methods work, rather than measure their real-world security.

• To demonstrate basic cryptanalysis by generating possible decryptions.

The analysis feature generates candidate results for selected methods. It tries all 26 shifts for Caesar, applies Atbash, checks valid Affine key combinations, and tries a range of Rail Fence key values. For Vigenère and Columnar Transposition, it uses candidate keywords entered by the user. The feature demonstrates how possible keys can be investigated, while making clear that generated candidates may not identify the original message.

• To apply Python programming concepts in an interactive project.

The application provides practical experience with functions, loops, conditional statements, strings, modular arithmetic, input validation, and event-driven programming with Tkinter. It also aims to organize the cipher operations into separate functions so the code is easier to follow and maintain.

• To explain the educational purpose and limitations of classical ciphers.

The project aims to show how keys control text transformations and how encryption can be reversed through decryption. It also explains that these classical ciphers have weaknesses and are included for learning purposes only. The application should not be used to protect confidential information.

4 Methodology

4.1 Architectural Diagram

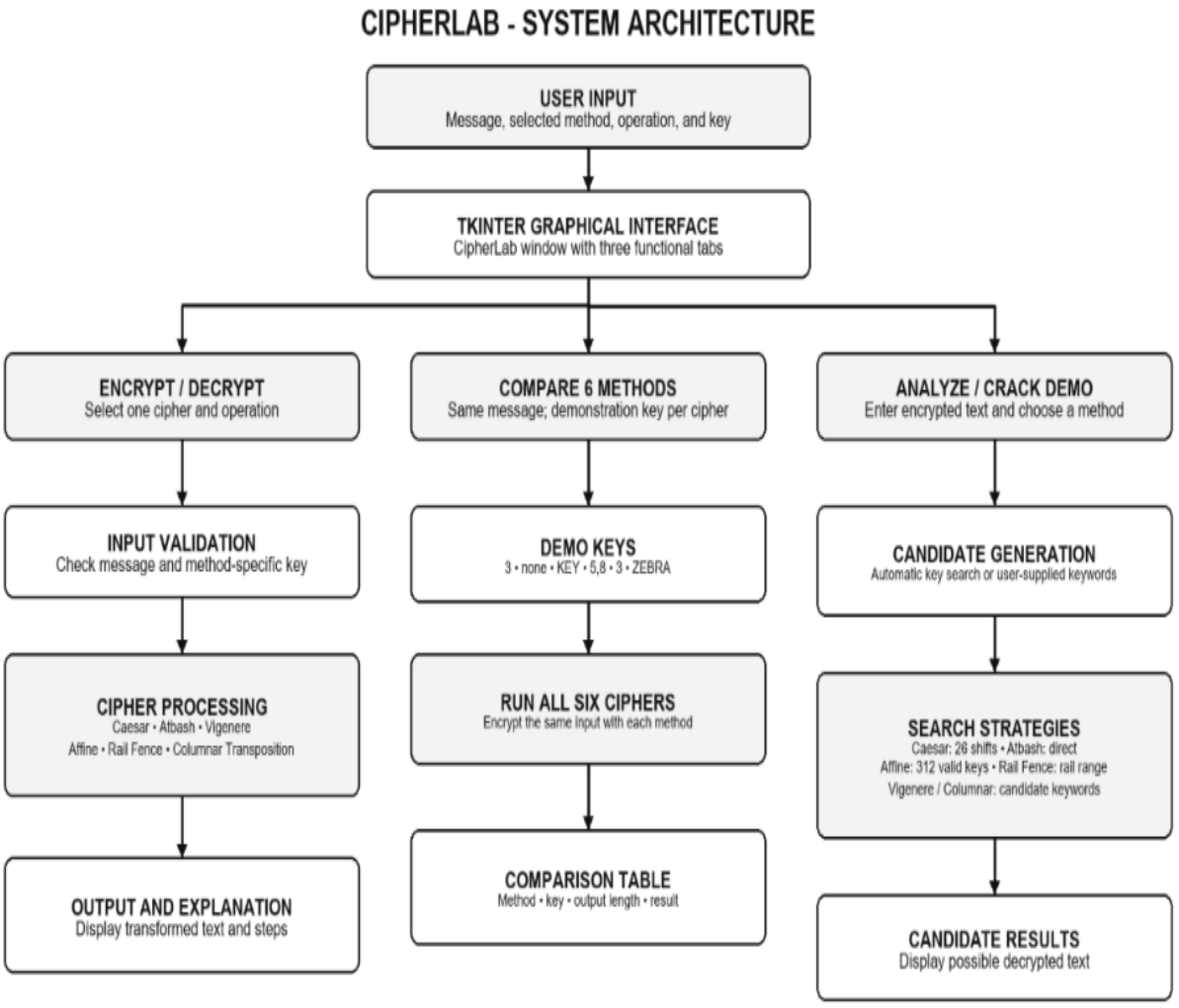


Figure 1: Architectural Diagram

The user enters a message and selects an operation, cipher, and key through the Tkinter interface. The program validates the message and checks that the key matches the selected method. If the input is invalid, it displays an error message; otherwise, it passes the information to the appropriate cipher function.

The **Encrypt / Decrypt** tab processes text using Caesar, Atbash, Vigenère, Affine, Rail Fence, or Columnar Transposition. It displays the transformed text and a step-by-step explanation. The **Compare 6 Methods** tab encrypts the same input with each method's demonstration key and presents the keys and outputs in a comparison table.

The **Analyse / Crack Demo** tab generates possible decryptions using the selected method. It searches defined key options for Caesar, Atbash, Affine, and Rail Fence, and tries user-supplied keywords for Vigenère and Columnar Transposition. The results are candidates for the user to

examine; the program does not guarantee it will identify the original text.

4.2 Hardware and Software Requirements

Hardware and Software Requirements

Hardware Requirements

- Desktop or laptop computer with a dual-core processor or better
- Minimum 4 GB RAM
- At least 100 MB of available storage
- Keyboard, mouse or touchpad, and display

Software Requirements

- Python 3.8 or later
- Tkinter library for the graphical interface
- Windows, macOS, or Linux with desktop GUI support
- Any Python editor or IDE, such as IDLE, Visual Studio Code, or PyCharm

4.3 Working Principle

CipherLab is a Python application with a Tkinter graphical interface. It has three tabs: **Encrypt / Decrypt**, **Compare 6 Methods**, and **Analyse / Crack Demo**. The user enters a message, selects a cipher and operation, and supplies the required key. The program validates the input before calling the selected cipher function. If a required value is missing or its format is invalid, the application displays an error message.

The **Encrypt / Decrypt** tab applies one of six methods. Caesar shifts English letters by a whole-number key, wrapping around the alphabet with modulo 26. Atbash replaces each letter with its reverse-alphabet partner and uses the same operation for encryption and decryption. Vigenère shifts each English letter according to a repeating keyword; punctuation, spaces, and digits do not advance the keyword. Affine maps each letter value using \(E(x) = (a \times x + b) \bmod 26\). For this method, \(a\) must be coprime with 26 so that it has an inverse for decryption.

Rail Fence and Columnar Transposition rearrange character positions rather than substitute letters. Rail Fence places characters in a zigzag across the selected number of rails and reads the rails in order; decryption rebuilds the zigzag pattern. Columnar Transposition writes the text across rows and reads columns in the alphabetical order of the keyword; decryption restores the row-wise arrangement. Since these methods rearrange the whole text, spaces and punctuation can move to new positions.

After processing the message, the program displays the result and a step-by-step explanation. The comparison tab encrypts the same input with all six methods using demonstration keys, then shows the key, output length, and result for each method in a table. This lets the user compare how each method transforms the same text.

The analysis tab generates possible decryptions for a selected method. It tries all 26 shifts for Caesar, applies Atbash directly, checks 312 valid Affine key combinations, and tries Rail Fence rail counts up to a limit of 12. For Vigenère and Columnar Transposition, the user supplies candidate keywords for the program to try. The resulting text is displayed as possible candidates; the program does not guarantee that any candidate is the original message. CipherLab is an educational demonstration, and its classical ciphers should not be used to protect confidential information.

Example Problem

Enter **HELLO** as the message. Compare the six encryption methods using the demonstration keys shown in the program.

Method	**Key**	**Encrypted text**	**Decrypted text**
Caesar	3	KHOOR	HELLO
Atbash	None	SVOOL	HELLO
Vigenère	KEY	RIJVS	HELLO
Affine	a = 5, b = 8	RCLLA	HELLO
Rail Fence	3 rails	HOELL	HELLO
Columnar Transposition	ZEBRA	OLELH	HELLO

The program's **Compare 6 Methods** tab produces the encrypted outputs for the same input message using these demonstration keys. To check decryption, select each method, enter its key, choose **Decrypt**, and enter that method's encrypted text. Each method should recover the original message, **HELL**

4.4 software Implementation

```
import math
import re
import tkinter as tk
from tkinter import messagebox, ttk

METHODS = (
    "Caesar",
    "Atbash",
    "Vigenere",
    "Affine",
    "Rail Fence",
    "Columnar Transposition",
)

DEMO_KEYS = {
    "Caesar": "3",
    "Atbash": "",
    "Vigenere": "KEY",
    "Affine": "5,8",
    "Rail Fence": "3",
    "Columnar Transposition": "ZEBRA",
}

METHOD_NOTES = {
    "Caesar": "Shifts each English letter by a fixed whole-number
key.",
    "Atbash": "Maps each English letter to its reverse-alphabet
partner; no key is needed.",
    "Vigenere": "Uses a repeating alphabetic keyword to vary the
letter shifts.",
    "Affine": "Uses E(x) = (a*x + b) mod 26; a must be coprime
with 26.",
    "Rail Fence": "Reorders characters along a zigzag pattern
using a rail-count key.",
    "Columnar Transposition": "Writes text in rows, then reads
columns in keyword order.",
}

def is_ascii_keyword(keyword):
    return bool(keyword) and all("A" <= char.upper() <= "Z" and
len(char.upper()) == 1 for char in keyword)

def caesar_cipher(text, key, decrypt=False):
    shift = -key if decrypt else key
    result = []
```

```
    for char in text:
        if "A" <= char <= "Z":
            result.append(chr((ord(char) - 65 + shift) % 26 + 65))
        elif "a" <= char <= "z":
            result.append(chr((ord(char) - 97 + shift) % 26 + 97))
        else:
            result.append(char)
    return "".join(result)

def atbash_cipher(text):
    result = []
    for char in text:
        if "A" <= char <= "Z":
            result.append(chr(90 - (ord(char) - 65)))
        elif "a" <= char <= "z":
            result.append(chr(122 - (ord(char) - 97)))
        else:
            result.append(char)
    return "".join(result)

def vigenere_cipher(text, keyword, decrypt=False):
    keyword = keyword.upper()
    result = []
    key_position = 0
    for char in text:
        if "A" <= char <= "Z" or "a" <= char <= "z":
            shift = ord(keyword[key_position % len(keyword)]) - 65
            if decrypt:
                shift = -shift
            base = 65 if char.isupper() else 97
            result.append(chr((ord(char) - base + shift) % 26 +
base))
            key_position += 1
        else:
            result.append(char)
    return "".join(result)

def affine_cipher(text, a, b, decrypt=False):
    a %= 26
    b %= 26
    if math.gcd(a, 26) != 1:
        raise ValueError("The Affine value a must be coprime with
26.")
    if decrypt:
        inverse_a = pow(a, -1, 26)
        convert = lambda x: (inverse_a * (x - b)) % 26
    else:
        convert = lambda x: (a * x + b) % 26
```

```
    result = []
    for char in text:
        if "A" <= char <= "Z":
            result.append(chr(convert(ord(char) - 65) + 65))
        elif "a" <= char <= "z":
            result.append(chr(convert(ord(char) - 97) + 97))
        else:
            result.append(char)
    return "".join(result)

def rail_pattern(length, rails):
    pattern = []
    rail = 0
    direction = 1
    for _ in range(length):
        pattern.append(rail)
        if rail == 0:
            direction = 1
        elif rail == rails - 1:
            direction = -1
        rail += direction
    return pattern

def rail_fence_cipher(text, rails, decrypt=False):
    if rails < 2:
        raise ValueError("Rail Fence needs at least 2 rails.")
    pattern = rail_pattern(len(text), rails)

    if not decrypt:
        fence = [[] for _ in range(rails)]
        for char, rail in zip(text, pattern):
            fence[rail].append(char)
        return "".join("".join(row) for row in fence)

    counts = [pattern.count(rail) for rail in range(rails)]
    fence = []
    offset = 0
    for count in counts:
        fence.append(list(text[offset:offset + count]))
        offset += count

    rail_positions = [0] * rails
    result = []
    for rail in pattern:
        result.append(fence[rail][rail_positions[rail]])
        rail_positions[rail] += 1
    return "".join(result)
```

```
def columnar_cipher(text, keyword, decrypt=False):
   keyword = keyword.upper()
   count = len(keyword)
   order = sorted(range(count), key=lambda index:
(keyword[index], index))

   if not decrypt:
      return "".join(text[column::count] for column in order)

   # In a ragged row-wise grid, the first remainder columns are
one character longer.
   base_length, remainder = divmod(len(text), count)
   columns = [""] * count
   offset = 0
   for column in order:
      length = base_length + (1 if column < remainder else 0)
      columns[column] = text[offset:offset + length]
      offset += length

   result = []
   rows = base_length + (1 if remainder else 0)
   for row in range(rows):
      for column in range(count):
         if row < len(columns[column]):
            result.append(columns[column][row])
   return "".join(result)

def parse_key(method, raw_key):
   key = raw_key.strip()
   if method == "Atbash":
      return None
   if not key:
      raise ValueError("Enter a key for the selected method.")

   if method == "Caesar":
      try:
         return int(key)
      except ValueError as exc:
         raise ValueError("The Caesar key must be a whole
number, such as 3.") from exc

   if method == "Atbash":
      return None

   if method == "Vigenere":
      if not is_ascii_keyword(key):
         raise ValueError("Use English letters A-Z only for the
Vigenere keyword.")
```

```
        return key

    if method == "Affine":
        pieces = [part.strip() for part in key.split(",")]
        if len(pieces) != 2:
            raise ValueError("Enter Affine key values as a,b, for
example 5,8.")
        try:
            a, b = (int(part) for part in pieces)
        except ValueError as exc:
            raise ValueError("Both Affine key values must be whole
numbers, such as 5,8.") from exc
        if math.gcd(a, 26) != 1:
            raise ValueError("Affine value a must be coprime with
26. Examples: 1, 3, 5, 7.")
        return a, b

    if method == "Rail Fence":
        try:
            rails = int(key)
        except ValueError as exc:
            raise ValueError("The Rail Fence key must be a whole
number of rails.") from exc
        if rails < 2:
            raise ValueError("The Rail Fence key must be at least
2.")
        return rails

    if method == "Columnar Transposition":
        if not is_ascii_keyword(key):
            raise ValueError("Use English letters A-Z only for the
column keyword.")
        return key

    raise ValueError("Choose one of the listed methods.")

def apply_cipher(method, text, key, decrypt=False):
    if method == "Caesar":
        return caesar_cipher(text, key, decrypt)
    if method == "Atbash":
        return atbash_cipher(text)
    if method == "Vigenere":
        return vigenere_cipher(text, key, decrypt)
    if method == "Affine":
        return affine_cipher(text, key[0], key[1], decrypt)
    if method == "Rail Fence":
        return rail_fence_cipher(text, key, decrypt)
    if method == "Columnar Transposition":
        return columnar_cipher(text, key, decrypt)
```

```
    raise ValueError("Choose one of the listed methods.")

def key_description(method, key):
    if method == "Atbash":
        return "none"
    if method == "Affine":
        return f"a={key[0]}, b={key[1]}"
    return str(key)

def explain_steps(method, text, key, decrypt, result):
    action = "Decryption" if decrypt else "Encryption"
    lines = [f"{action} steps - {method}", METHOD_NOTES[method],
""]

    if method == "Caesar":
        shift = -key if decrypt else key
        lines.append(f"Effective shift: {shift} (wrapped into 0-
25 with modulo 26).")
        lines.append("Character changes:")
        for index, char in enumerate(text):
            changed = caesar_cipher(char, key, decrypt)
            if char.isascii() and char.isalpha():
                lines.append(f"  {index + 1}. {char} ->
{changed}  (shift {shift})")
            else:
                lines.append(f"  {index + 1}. {char!r} ->
{changed!r}  (unchanged)")

    elif method == "Atbash":
        lines.append("Each English letter is reflected across the
alphabet: A <-> Z, B <-> Y, and so on.")
        lines.append("Character changes:")
        for index, char in enumerate(text):
            changed = atbash_cipher(char)
            status = "mapped" if char.isascii() and char.isalpha()
else "unchanged"
            lines.append(f"  {index + 1}. {char!r} ->
{changed!r}  ({status})")
        lines.append("The same operation encrypts and decrypts
Atbash text.")

    elif method == "Vigenere":
        lines.append(f"Keyword: {key.upper()}. Advance to the
next keyword letter only for an English letter.")
        lines.append("Character changes:")
        position = 0
        for index, char in enumerate(text):
            if char.isascii() and char.isalpha():
                key_char = key.upper()[position % len(key)]
```

```
            shift = ord(key_char) - 65
            if decrypt:
               shift = -shift
            changed = vigenere_cipher(char, key, decrypt)
            lines.append(f"  {index + 1}. {char} with {key_char}
(shift {shift}) -> {changed}")
            position += 1
         else:
            lines.append(f"  {index + 1}. {char!r} ->
{char!r}  (unchanged; keyword does not advance)")

   elif method == "Affine":
      a, b = key
      lines.append(f"Key: a={a}, b={b}; a modulo 26 must have a
multiplicative inverse.")
      if decrypt:
         lines.append(f"Decryption uses the modular inverse of
a={a % 26}, then subtracts b={b % 26}.")
      else:
         lines.append("Encryption formula: E(x) = (a*x + b) mod
26, where A=0 through Z=25.")
      lines.append("Character changes:")
      for index, char in enumerate(text):
         changed = affine_cipher(char, a, b, decrypt)
         if char.isascii() and char.isalpha():
            lines.append(f"  {index + 1}. {char} -> {changed}")
         else:
            lines.append(f"  {index + 1}. {char!r} ->
{char!r}  (unchanged)")

   elif method == "Rail Fence":
      pattern = rail_pattern(len(text), key)
      lines.append(f"The message follows a zigzag across {key}
rails.")
      lines.append("Character positions and assigned rails:")
      for index, (char, rail) in enumerate(zip(text, pattern)):
         lines.append(f"  Position {index + 1}: {char!r} -> rail
{rail + 1}")
      if decrypt:
         lines.append("The ciphertext is split into rail
lengths, then read back along the zigzag.")
      else:
         lines.append("The rails are read from top to bottom to
form the encrypted text.")

   elif method == "Columnar Transposition":
      keyword = key.upper()
      order = sorted(range(len(keyword)), key=lambda index:
(keyword[index], index))
```

```
      lines.append(f"Keyword: {keyword}. Columns are read in
alphabetical keyword order.")
      lines.append("Read order: " + ", ".join(f"column {i + 1}
({keyword[i]})" for i in order))
      if decrypt:
         lines.append("The ciphertext is split into columns in
that order, then read across the rows.")
      else:
         lines.append("The text is written left-to-right across
rows, then read in the listed column order.")
      lines.append("Grid:")
      width = len(keyword)
      if decrypt:
         grid_text = text
         row_count = (len(grid_text) + width - 1) // width
         base, rem = divmod(len(grid_text), width)
         col_lengths = [base + (1 if col < rem else 0) for col
in range(width)]
         cols = [""] * width
         offset = 0
         for col in order:
            cols[col] = grid_text[offset:offset +
col_lengths[col]]
            offset += col_lengths[col]
         for row in range(row_count):
            lines.append("  " + "  ".join(cols[col][row] if row
< len(cols[col]) else " " for col in range(width)))
      else:
         for start in range(0, len(text), width):
            row = text[start:start + width]
            lines.append("  " +
"  ".join(row.ljust(width)[:width]))

   lines.extend(["", f"Final output: {result}"])
   return "\n".join(lines)

def write_text(widget, content, readonly=False):
   widget.configure(state="normal")
   widget.delete("1.0", tk.END)
   widget.insert("1.0", content)
   if readonly:
      widget.configure(state="disabled")

def add_scrolled_text(parent, height, font=("Consolas", 10),
readonly=False):
   wrap = ttk.Frame(parent)
   wrap.rowconfigure(0, weight=1)
   wrap.columnconfigure(0, weight=1)
   text = tk.Text(
```

```
        wrap,
        height=height,
        wrap="word",
        font=font,
        bg="#FFFFFF",
        fg="#17243A",
        insertbackground="#17243A",
        relief="flat",
        padx=10,
        pady=9,
        undo=not readonly,
    )
    ybar = ttk.Scrollbar(wrap, orient="vertical",
command=text.yview)
    text.configure(yscrollcommand=ybar.set)
    text.grid(row=0, column=0, sticky="nsew")
    ybar.grid(row=0, column=1, sticky="ns")
    if readonly:
        text.configure(state="disabled")
    return wrap, text

def make_tree(parent, columns, headings, widths):
    frame = ttk.Frame(parent)
    frame.rowconfigure(0, weight=1)
    frame.columnconfigure(0, weight=1)
    tree = ttk.Treeview(frame, columns=columns, show="headings",
height=12)
    for col, heading, width in zip(columns, headings, widths):
        tree.heading(col, text=heading)
        tree.column(col, width=width, minwidth=80, anchor="w",
stretch=True)
    ybar = ttk.Scrollbar(frame, orient="vertical",
command=tree.yview)
    xbar = ttk.Scrollbar(frame, orient="horizontal",
command=tree.xview)
    tree.configure(yscrollcommand=ybar.set,
xscrollcommand=xbar.set)
    tree.grid(row=0, column=0, sticky="nsew")
    ybar.grid(row=0, column=1, sticky="ns")
    xbar.grid(row=1, column=0, sticky="ew")
    tree.tag_configure("odd", background="#F2F6FB")
    return frame, tree

class CipherLearningSuite:
    def __init__(self, root):
        self.root = root
        root.title("CipherLab | Text Encryption and Decryption")
        root.geometry("1120x880")
```

```
        root.minsize(900, 700)
        root.configure(bg="#F3F6FA")

        style = ttk.Style()
        style.theme_use("clam")
        style.configure("TFrame", background="#F3F6FA")
        style.configure("Card.TFrame", background="#FFFFFF")
        style.configure("TLabel", background="#F3F6FA",
foreground="#1B2B45", font=("Segoe UI", 10))
        style.configure("Card.TLabel", background="#FFFFFF",
foreground="#1B2B45", font=("Segoe UI", 10))
        style.configure("Title.TLabel", background="#17365D",
foreground="#FFFFFF", font=("Segoe UI", 21, "bold"))
        style.configure("Subtitle.TLabel", background="#17365D",
foreground="#DCE8F5", font=("Segoe UI", 10))
        style.configure("Section.TLabel", background="#FFFFFF",
foreground="#17365D", font=("Segoe UI", 12, "bold"))
        style.configure("TButton", font=("Segoe UI", 10),
padding=(11, 7))
        style.configure("Accent.TButton", background="#2367A5",
foreground="#FFFFFF", font=("Segoe UI", 10, "bold"),
padding=(12, 8))
        style.map("Accent.TButton", background=[("active",
"#1A5388")])
        style.configure("TNotebook", background="#F3F6FA",
borderwidth=0)
        style.configure("TNotebook.Tab", padding=(15, 9),
font=("Segoe UI", 10, "bold"))
        style.configure("Treeview", font=("Segoe UI", 9),
rowheight=27, background="#FFFFFF", fieldbackground="#FFFFFF")
        style.configure("Treeview.Heading", font=("Segoe UI", 9,
"bold"), background="#DCE8F5", foreground="#17365D")

        header = tk.Frame(root, bg="#17365D", padx=24, pady=18)
        header.pack(fill="x")
        ttk.Label(header, text="CipherLab",
style="Title.TLabel").pack(anchor="w")
        ttk.Label(
            header,
            text="Explore six classical ciphers, compare their
outputs, and inspect possible decryptions.",
            style="Subtitle.TLabel",
        ).pack(anchor="w", pady=(3, 0))

        shell = ttk.Frame(root, padding=(18, 14, 18, 10))
        shell.pack(fill="both", expand=True)
        self.tabs = ttk.Notebook(shell)
        self.tabs.pack(fill="both", expand=True)
```

```
        self.build_transform_tab()
        self.build_compare_tab()
        self.build_analyze_tab()

        footer = tk.Frame(root, bg="#E5ECF4", padx=18, pady=8)
        footer.pack(fill="x", side="bottom")
        tk.Label(
            footer,
            text="Learning demonstration only - classical ciphers
are not suitable for confidential data.",
            bg="#E5ECF4",
            fg="#52657A",
            font=("Segoe UI", 9),
        ).pack(anchor="w")

    def card(self, parent, padding=14):
        return ttk.Frame(parent, style="Card.TFrame",
padding=padding)

    def build_transform_tab(self):
        tab = ttk.Frame(self.tabs, padding=12)
        tab.columnconfigure(0, weight=1)
        tab.rowconfigure(1, weight=1)
        tab.rowconfigure(3, weight=1)
        tab.rowconfigure(4, weight=2)
        self.tabs.add(tab, text="Encrypt / Decrypt")

        controls = self.card(tab)
        controls.grid(row=0, column=0, sticky="ew", pady=(0, 10))
        controls.columnconfigure(5, weight=1)
        ttk.Label(controls, text="Method",
style="Card.TLabel").grid(row=0, column=0, sticky="w")
        self.method_var = tk.StringVar(value="Caesar")
        method_menu = ttk.Combobox(controls,
textvariable=self.method_var, values=METHODS, state="readonly",
width=25)
        method_menu.grid(row=1, column=0, sticky="w", padx=(0,
14), pady=(5, 0))
        method_menu.bind("<<ComboboxSelected>>",
self.update_key_hint)

        ttk.Label(controls, text="Operation",
style="Card.TLabel").grid(row=0, column=1, sticky="w")
        self.operation_var = tk.StringVar(value="Encrypt")
        ttk.Combobox(controls, textvariable=self.operation_var,
values=("Encrypt", "Decrypt"), state="readonly",
width=13).grid(row=1, column=1, sticky="w", padx=(0, 14),
pady=(5, 0))
```

```
        self.key_label = ttk.Label(controls, text="Key (whole
number)", style="Card.TLabel")
        self.key_label.grid(row=0, column=2, sticky="w")
        self.key_entry = ttk.Entry(controls, width=20)
        self.key_entry.grid(row=1, column=2, sticky="w", padx=(0,
8), pady=(5, 0))
        self.key_hint = ttk.Label(controls, text="Example: 3",
style="Card.TLabel")
        self.key_hint.grid(row=1, column=3, sticky="w", pady=(5,
0))

        message_card = self.card(tab)
        message_card.grid(row=1, column=0, sticky="ew", pady=(0,
10))
        message_card.columnconfigure(0, weight=1)
        ttk.Label(message_card, text="Your message",
style="Section.TLabel").grid(row=0, column=0, sticky="w",
pady=(0, 7))
        message_wrap, self.message_text =
add_scrolled_text(message_card, 6)
        message_wrap.grid(row=1, column=0, sticky="ew")

        button_row = ttk.Frame(tab)
        button_row.grid(row=2, column=0, sticky="ew", pady=(0,
10))
        ttk.Button(button_row, text="Process Message",
style="Accent.TButton",
command=self.process_message).pack(side="left")
        ttk.Button(button_row, text="Clear",
command=self.clear_transform).pack(side="left", padx=8)

        output_card = self.card(tab)
        output_card.grid(row=3, column=0, sticky="nsew", pady=(0,
10))
        output_card.columnconfigure(0, weight=1)
        output_card.rowconfigure(1, weight=1)
        ttk.Label(output_card, text="Result",
style="Section.TLabel").grid(row=0, column=0, sticky="w",
pady=(0, 7))
        result_wrap, self.result_text =
add_scrolled_text(output_card, 5, readonly=True)
        result_wrap.grid(row=1, column=0, sticky="nsew")

        steps_card = self.card(tab)
        steps_card.grid(row=4, column=0, sticky="nsew")
        steps_card.columnconfigure(0, weight=1)
        steps_card.rowconfigure(1, weight=1)
        ttk.Label(steps_card, text="How it worked",
style="Section.TLabel").grid(row=0, column=0, sticky="w",
```

```
pady=(0, 7))
        steps_wrap, self.steps_text =
add_scrolled_text(steps_card, 9, readonly=True)
        steps_wrap.grid(row=1, column=0, sticky="nsew")
        self.update_key_hint()

    def update_key_hint(self, _event=None):
        method = self.method_var.get()
        values = {
            "Caesar": ("Key (whole number)", "Example: 3", True),
            "Atbash": ("Key", "No key required", False),
            "Vigenere": ("Keyword (A-Z)", "Example: KEY", True),
            "Affine": ("Key (a,b)", "Example: 5,8", True),
            "Rail Fence": ("Key (rails)", "Example: 3", True),
            "Columnar Transposition": ("Keyword (A-Z)", "Example:
ZEBRA", True),
        }
        label, hint, enabled = values[method]
        self.key_label.configure(text=label)
        self.key_hint.configure(text=hint)
        self.key_entry.configure(state="normal" if enabled else
"disabled")

    def process_message(self):
        message = self.message_text.get("1.0",
tk.END).rstrip("\n")
        if not message.strip():
            messagebox.showerror("Message required", "Enter a
message first.")
            return
        method = self.method_var.get()
        decrypt = self.operation_var.get() == "Decrypt"
        try:
            key = parse_key(method, self.key_entry.get())
            output = apply_cipher(method, message, key, decrypt)
        except ValueError as exc:
            messagebox.showerror("Check the key", str(exc))
            return

        title = "Decrypted text" if decrypt else "Encrypted text"
        write_text(self.result_text, f"Method:
{method}\n{title}:\n{output}", readonly=True)
        write_text(self.steps_text, explain_steps(method,
message, key, decrypt, output), readonly=True)

    def clear_transform(self):
        self.message_text.delete("1.0", tk.END)
        self.key_entry.configure(state="normal")
        self.key_entry.delete(0, tk.END)
```

```
        self.method_var.set("Caesar")
        self.operation_var.set("Encrypt")
        self.update_key_hint()
        write_text(self.result_text, "", readonly=True)
        write_text(self.steps_text, "", readonly=True)

    def build_compare_tab(self):
        tab = ttk.Frame(self.tabs, padding=12)
        tab.columnconfigure(0, weight=1)
        tab.rowconfigure(1, weight=1)
        self.tabs.add(tab, text="Compare 6 Methods")

        card = self.card(tab)
        card.grid(row=0, column=0, sticky="ew", pady=(0, 10))
        card.columnconfigure(0, weight=1)
        ttk.Label(card, text="Compare the same message",
style="Section.TLabel").grid(row=0, column=0, sticky="w",
pady=(0, 6))
        ttk.Label(card, text="The table uses a visible
demonstration key for each method.",
style="Card.TLabel").grid(row=1, column=0, sticky="w", pady=(0,
7))
        compare_wrap, self.compare_input =
add_scrolled_text(card, 5)
        compare_wrap.grid(row=2, column=0, sticky="ew")
        ttk.Button(card, text="Compare All Methods",
style="Accent.TButton",
command=self.compare_methods).grid(row=3, column=0, sticky="w",
pady=(9, 0))

        table_card = self.card(tab)
        table_card.grid(row=1, column=0, sticky="nsew")
        table_card.columnconfigure(0, weight=1)
        table_card.rowconfigure(1, weight=1)
        ttk.Label(table_card, text="Encryption comparison",
style="Section.TLabel").grid(row=0, column=0, sticky="w",
pady=(0, 7))
        tree_wrap, self.compare_tree = make_tree(
            table_card,
            ("method", "key", "length", "output"),
            ("Method", "Demo key", "Length", "Encrypted output"),
            (205, 120, 80, 600),
        )
        tree_wrap.grid(row=1, column=0, sticky="nsew")
        note = ttk.Label(tab, text="Outputs are for observing the
transformations; equal lengths or different-looking text do not
measure real security.", wraplength=1000)
        note.grid(row=2, column=0, sticky="w", pady=(8, 0))
```

```
   def compare_methods(self):
      message = self.compare_input.get("1.0",
tk.END).rstrip("\n")
      if not message.strip():
         messagebox.showerror("Message required", "Enter a
message to compare.")
         return
      for row in self.compare_tree.get_children():
         self.compare_tree.delete(row)
      try:
         for index, method in enumerate(METHODS):
            raw_key = DEMO_KEYS[method]
            key = parse_key(method, raw_key)
            result = apply_cipher(method, message, key, False)
            display_result = result.replace("\n",
"\\n").replace("\r", "\\r")
            shown_key = raw_key or "None"
            self.compare_tree.insert("", "end", values=(method,
shown_key, len(result), display_result), tags=("odd" if index %
2 else "",))
      except ValueError as exc:
         messagebox.showerror("Comparison error", str(exc))

   def build_analyze_tab(self):
      tab = ttk.Frame(self.tabs, padding=12)
      tab.columnconfigure(0, weight=1)
      tab.rowconfigure(1, weight=1)
      self.tabs.add(tab, text="Analyze / Crack Demo")

      card = self.card(tab)
      card.grid(row=0, column=0, sticky="ew", pady=(0, 10))
      card.columnconfigure(1, weight=1)
      ttk.Label(card, text="Generate possible decryptions",
style="Section.TLabel").grid(row=0, column=0, columnspan=2,
sticky="w", pady=(0, 8))
      ttk.Label(card, text="Cipher method",
style="Card.TLabel").grid(row=1, column=0, sticky="w", padx=(0,
8))
      self.analyze_method = tk.StringVar(value="Caesar")
      analyze_menu = ttk.Combobox(card,
textvariable=self.analyze_method, values=METHODS,
state="readonly", width=27)
      analyze_menu.grid(row=1, column=1, sticky="w", pady=3)
      ttk.Label(card, text="Encrypted text",
style="Card.TLabel").grid(row=2, column=0, sticky="nw", padx=(0,
8), pady=5)
      cipher_wrap, self.cipher_input = add_scrolled_text(card,
4)
      cipher_wrap.grid(row=2, column=1, sticky="ew", pady=5)
```

```
      ttk.Label(card, text="Candidate keywords",
style="Card.TLabel").grid(row=3, column=0, sticky="w", padx=(0,
8), pady=5)
      self.candidate_entry = ttk.Entry(card)
      self.candidate_entry.insert(0, "KEY, LEMON, APPLE,
ZEBRA")
      self.candidate_entry.grid(row=3, column=1, sticky="ew",
pady=5)
      ttk.Label(
         card,
         text="Vigenere and Columnar try the comma-separated
keywords above. Caesar, Atbash, Affine, and Rail Fence generate
candidates automatically.",
         style="Card.TLabel",
         wraplength=900,
      ).grid(row=4, column=0, columnspan=2, sticky="w",
pady=(4, 8))
      ttk.Button(card, text="Generate Candidates",
style="Accent.TButton",
command=self.generate_candidates).grid(row=5, column=0,
columnspan=2, sticky="w")

      results_card = self.card(tab)
      results_card.grid(row=1, column=0, sticky="nsew")
      results_card.columnconfigure(0, weight=1)
      results_card.rowconfigure(1, weight=1)
      ttk.Label(results_card, text="Candidate decryptions",
style="Section.TLabel").grid(row=0, column=0, sticky="w",
pady=(0, 7))
      tree_wrap, self.candidate_tree = make_tree(
         results_card,
         ("key", "candidate"),
         ("Key tried", "Possible decrypted text"),
         (180, 800),
      )
      tree_wrap.grid(row=1, column=0, sticky="nsew")
      ttk.Label(
         tab,
         text="These are candidate outputs, not guaranteed
solutions. Long or unusual messages may need more candidate keys
or manual interpretation.",
         wraplength=1000,
      ).grid(row=2, column=0, sticky="w", pady=(8, 0))

   def generate_candidates(self):
      method = self.analyze_method.get()
      ciphertext = self.cipher_input.get("1.0",
tk.END).rstrip("\n")
      if not ciphertext.strip():
```

```
            messagebox.showerror("Encrypted text required", "Enter
text to analyze.")
            return

        candidates = []
        if method == "Caesar":
            candidates = [(str(key), caesar_cipher(ciphertext,
key, True)) for key in range(26)]
        elif method == "Atbash":
            candidates = [("no key", atbash_cipher(ciphertext))]
        elif method == "Affine":
            for a in range(26):
                if math.gcd(a, 26) == 1:
                    for b in range(26):
                        candidates.append((f"a={a}, b={b}",
affine_cipher(ciphertext, a, b, True)))
        elif method == "Rail Fence":
            # For rail counts at or above the text length, the
ordering is unchanged.
            max_rails = min(max(len(ciphertext), 2), 12)
            candidates = [(f"rails={rails}",
rail_fence_cipher(ciphertext, rails, True)) for rails in
range(2, max_rails + 1)]
        else:
            raw_candidates = [part.strip() for part in
re.split(r"[,;\n]+", self.candidate_entry.get()) if
part.strip()]
            if not raw_candidates:
                messagebox.showerror("Candidate keywords required",
"Enter one or more candidate keywords separated by commas.")
                return
            for keyword in raw_candidates:
                if not is_ascii_keyword(keyword):
                    messagebox.showerror("Invalid candidate
keyword", f"'{keyword}' must contain English letters A-Z only.")
                    return
                if method == "Vigenere":
                    result = vigenere_cipher(ciphertext, keyword,
True)
                else:
                    result = columnar_cipher(ciphertext, keyword,
True)
                candidates.append((f"keyword={keyword.upper()}",
result))

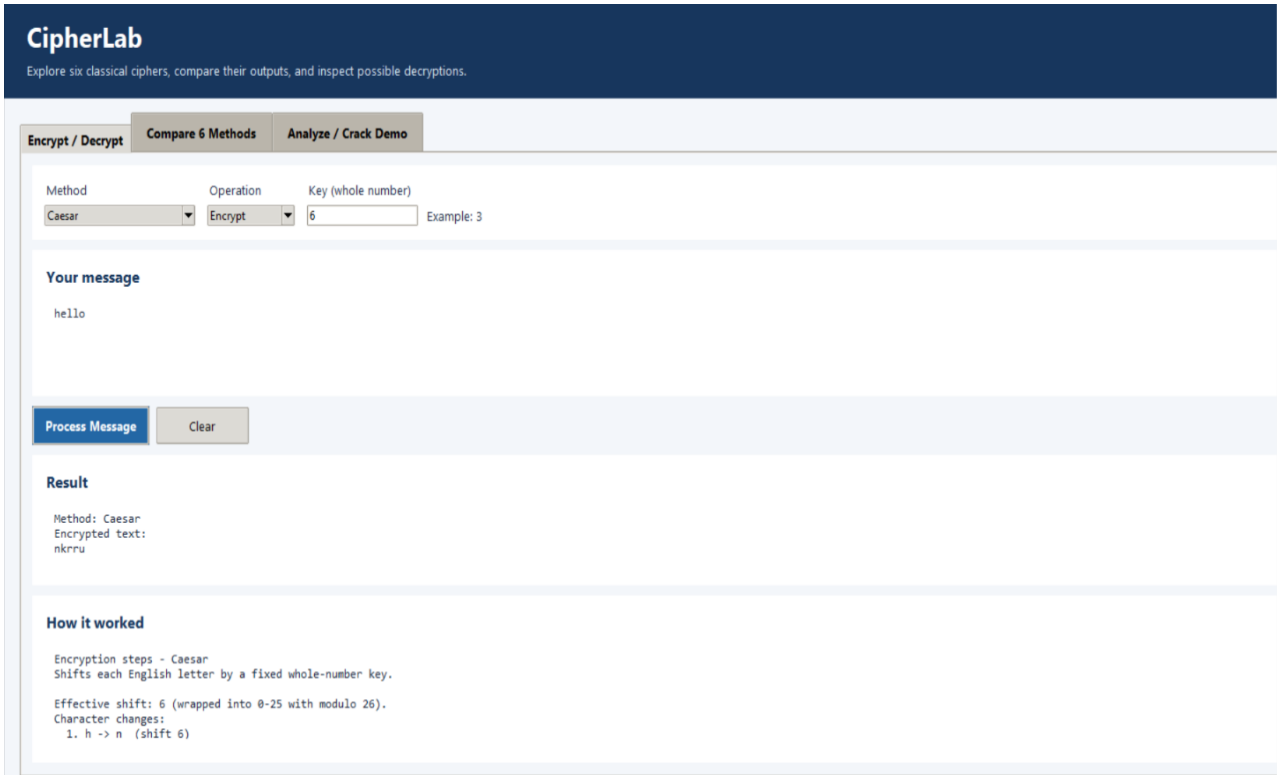
        for row in self.candidate_tree.get_children():
            self.candidate_tree.delete(row)
        for index, (key_text, candidate) in
enumerate(candidates):
```

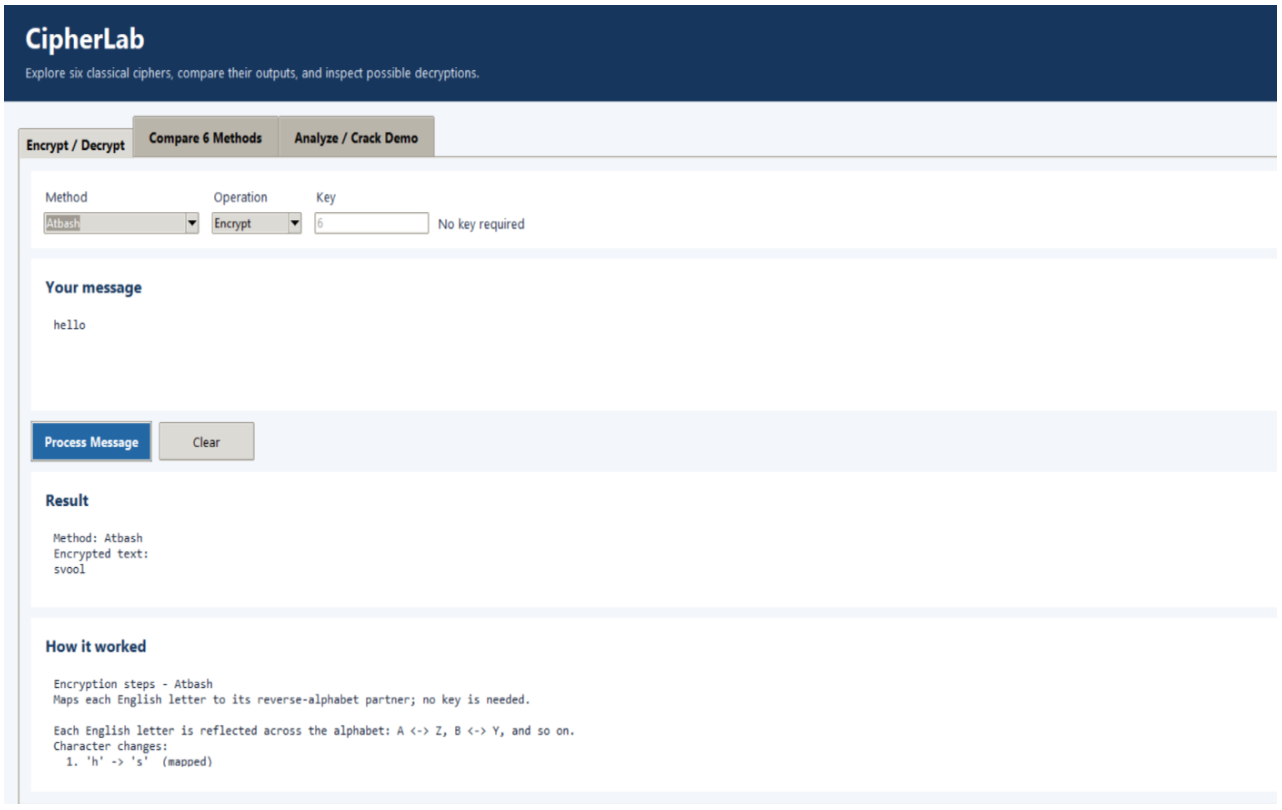
```
            display = candidate.replace("\n", "\\n").replace("\r",
"\\r")
            self.candidate_tree.insert("", "end",
values=(key_text, display), tags=("odd" if index % 2 else "",))

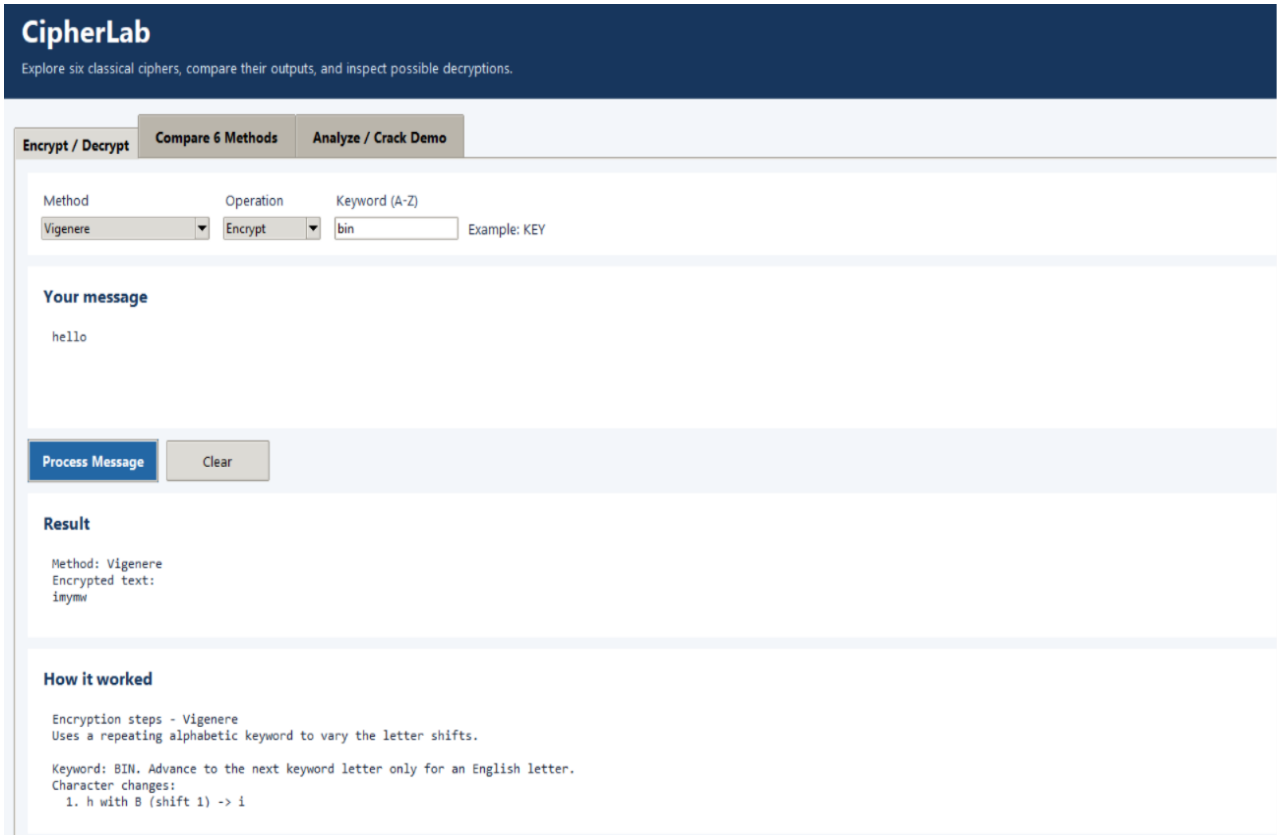
def main():
    root = tk.Tk()
    CipherLearningSuite(root)
    root.mainloop()

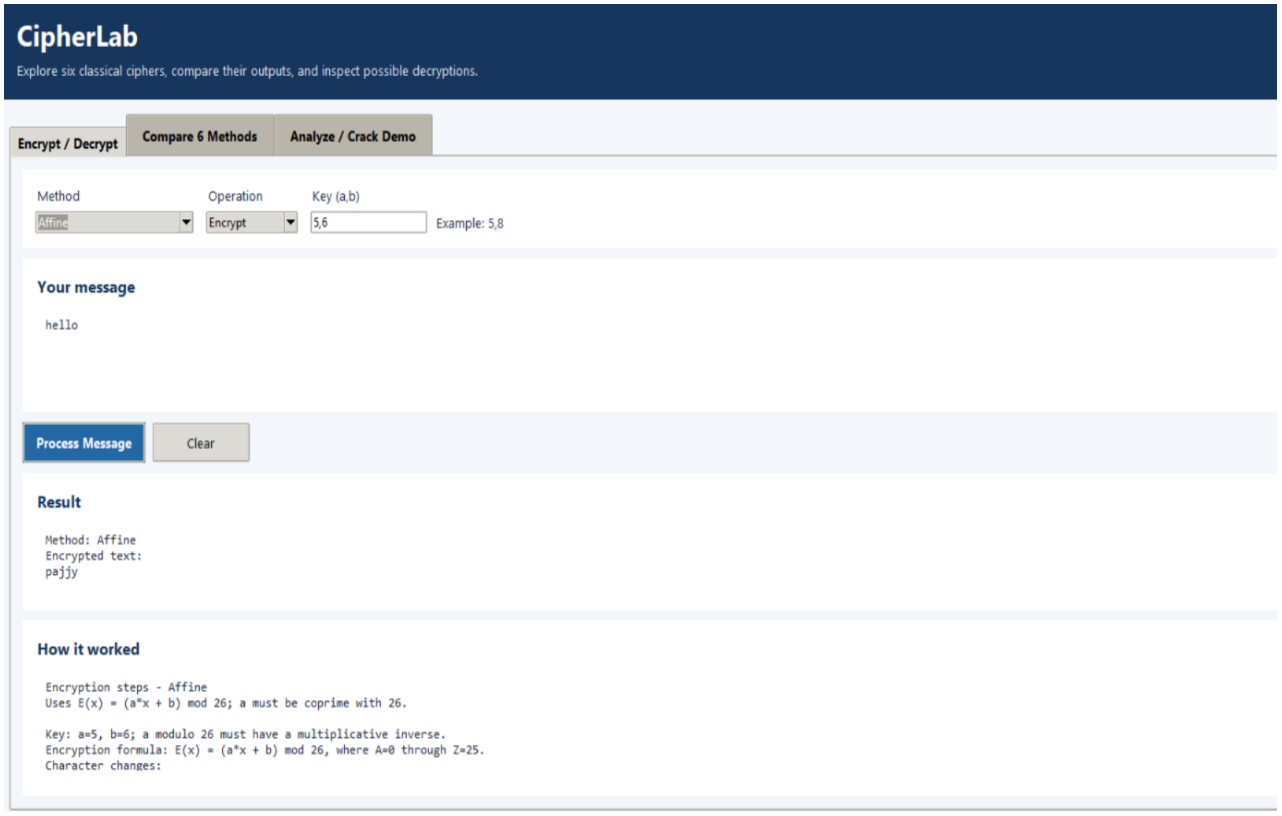
if __name__ == "__main__":
    main()
```

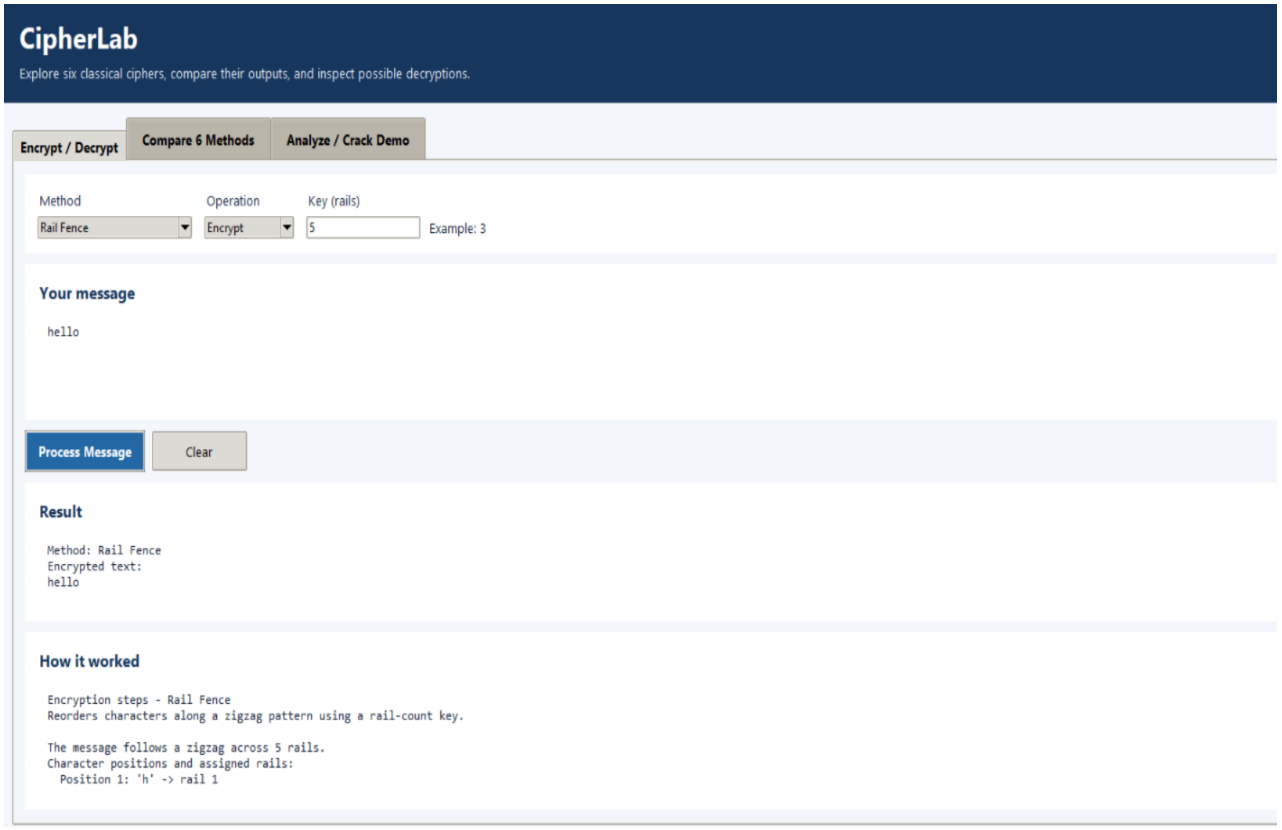
5 Results and Discussion

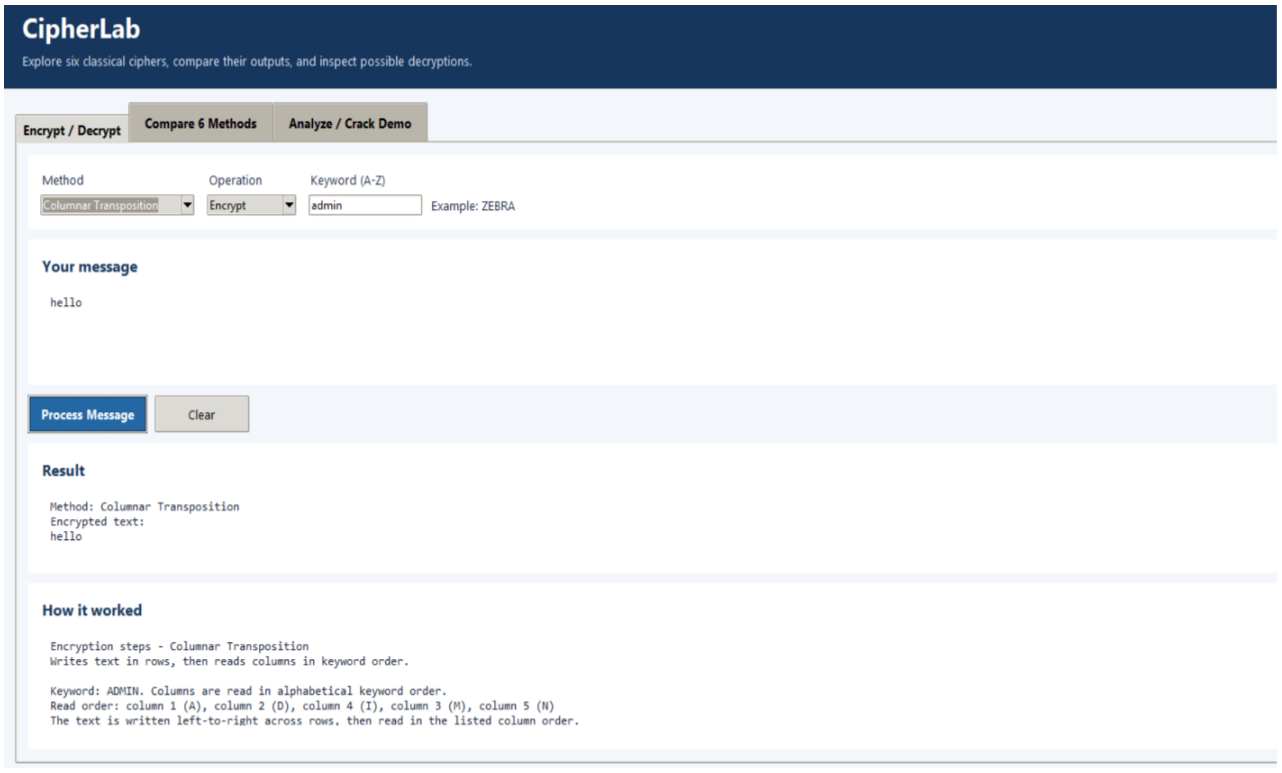


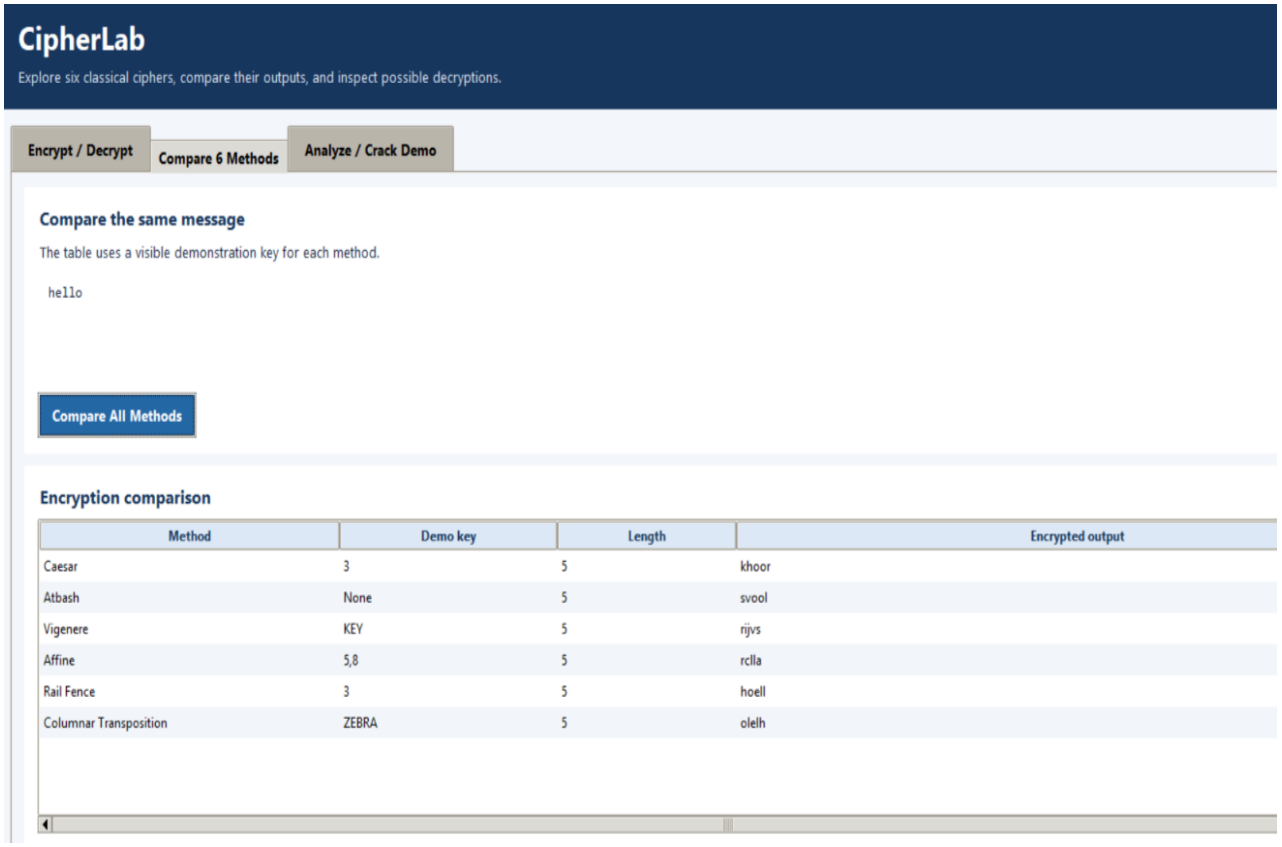


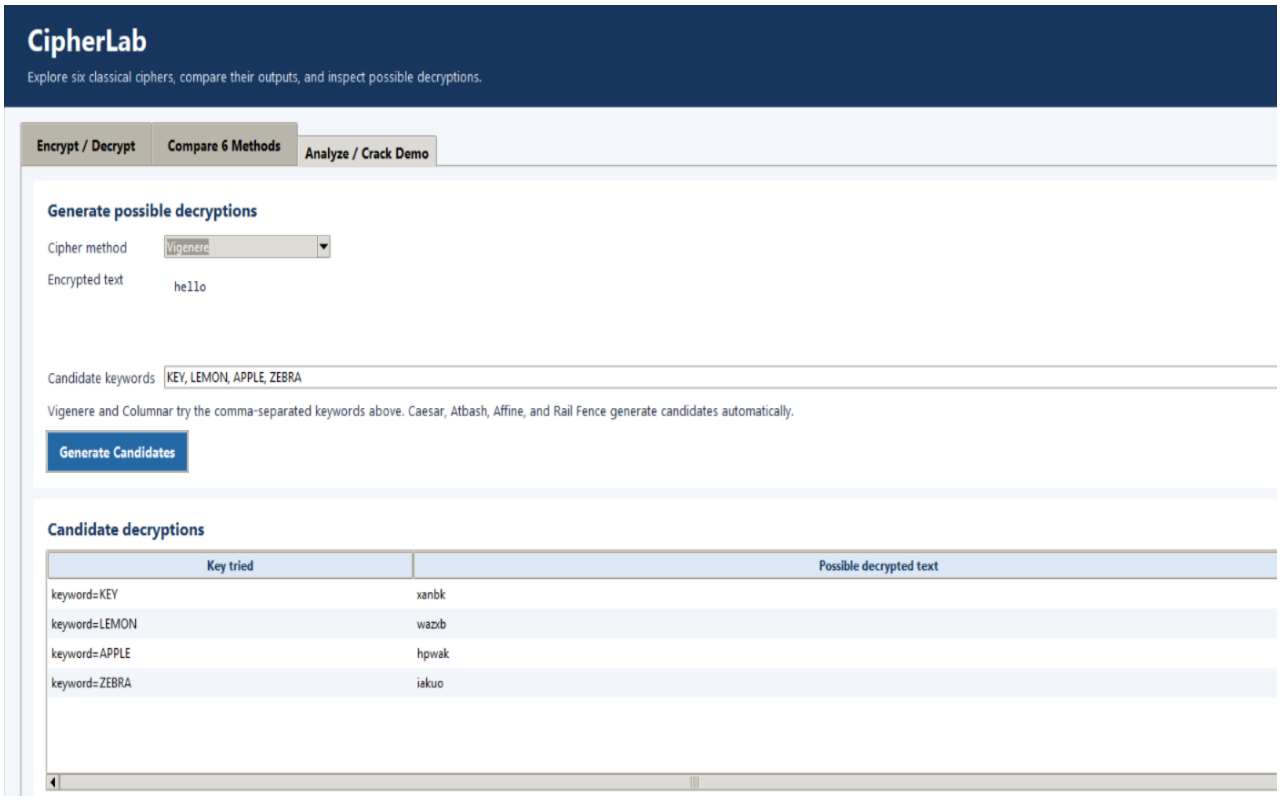

CipherLab
Explore six classical ciphers, compare their outputs, and inspect possible decryptions.
Encrypt / Decrypt
Compare 6 Methods
Analyze / Crack Demo
Method
Vigenere
Operation
Encrypt
Keyword (A-Z)
bin
Example: KEY
Your message
hello
Process Message
Clear
Result
Method: Vigenere
Encrypted text:
imymw
How it worked
Encryption steps - Vigenere
Uses a repeating alphabetic keyword to vary the letter shifts.
Keyword: BIN. Advance to the next keyword letter only for an English letter.
Character changes:
1. h with B (shift 1) -> i


CipherLab
Explore six classical ciphers, compare their outputs, and inspect possible decryptions.
Encrypt / Decrypt
Compare 6 Methods
Analyze / Crack Demo
Method
Affine
Operation
Encrypt
Key (a,b)
5,6
Example: 5,8
Your message
hello
Process Message
Clear
Result
Method: Affine
Encrypted text:
pajjy
How it worked
Encryption steps - Affine
Uses E(x) = (a*x + b) mod 26; a must be coprime with 26.
Key: a=5, b=6; a modulo 26 must have a multiplicative inverse.
Encryption formula: E(x) = (a*x + b) mod 26, where A=0 through Z=25.
Character changes:


CipherLab
Explore six classical ciphers, compare their outputs, and inspect possible decryptions.
Encrypt / Decrypt
Compare 6 Methods
Analyze / Crack Demo
Method
Rail Fence
Operation
Encrypt
Key (rails)
5
Example: 3
Your message
hello
Process Message
Clear
Result
Method: Rail Fence
Encrypted text:
hello
How it worked
Encryption steps - Rail Fence
Reorders characters along a zigzag pattern using a rail-count key.
The message follows a zigzag across 5 rails.
Character positions and assigned rails:
Position 1: 'h' -> rail 1


CipherLab
Explore six classical ciphers, compare their outputs, and inspect possible decryptions.
Encrypt / Decrypt
Compare 6 Methods
Analyze / Crack Demo
Method
Columnar Transposition
Operation
Encrypt
Keyword (A-Z)
admin
Example: ZEBRA
Your message
hello
Process Message
Clear
Result
Method: Columnar Transposition
Encrypted text:
hello
How it worked
Encryption steps - Columnar Transposition
Writes text in rows, then reads columns in keyword order.
Keyword: ADMIN. Columns are read in alphabetical keyword order.
Read order: column 1 (A), column 2 (D), column 4 (I), column 3 (M), column 5 (N)
The text is written left-to-right across rows, then read in the listed column order.


CipherLab
Explore six classical ciphers, compare their outputs, and inspect possible decryptions.
Encrypt / Decrypt
Compare 6 Methods
Analyze / Crack Demo
Compare the same message
The table uses a visible demonstration key for each method.
hello
Compare All Methods
Encryption comparison
Method
Demo key
Length
Encrypted output
Caesar
3
5
khoor
Atbash
None
5
svool
Vigenere
KEY
5
rijvs
Affine
5,8
rclla
Rail Fence
3
5
hoell
Columnar Transposition
ZEBRA
5
olelh


CipherLab
Explore six classical ciphers, compare their outputs, and inspect possible decryptions.
Encrypt / Decrypt
Compare 6 Methods
Analyze / Crack Demo
Generate possible decryptions
Cipher method
Vigenere
Encrypted text
hello
Candidate keywords
KEY, LEMON, APPLE, ZEBRA
Vigenere and Columnar try the comma-separated keywords above. Caesar, Atbash, Affine, and Rail Fence generate candidates automatically.
Generate Candidates
Candidate decryptions
Key tried
Possible decrypted text
keyword=KEY
xanbk
keyword=LEMON
wazxb
keyword=APPLE
hpwak
keyword=ZEBRA
iakuo

Summary of Results

CipherLab provides encryption and decryption using six methods: Caesar, Atbash, Vigenère, Affine, Rail Fence, and Columnar Transposition. For the sample message **HELLO**, the methods produce the following encrypted outputs using their demonstration keys:

Method	**Demonstration key**	**Encrypted output**
Caesar	3	KHOOR
Atbash	None	SVOOL
Vigenère	KEY	RIJVS
Affine	a = 5, b = 8	RCLLA
Rail Fence	3 rails	HOELL
Columnar Transposition	ZEBRA	OLELH

Decrypting each output with the corresponding method and key recovers **HELLO**. The comparison tab displays the six outputs for the same input, while the encryption and decryption tab shows the selected method's result and processing steps.

The analysis tab generates possible decryptions by trying available key options or user-supplied keywords. It displays these as candidates for inspection; it does not guarantee that it will identify the original message.

6 Mapping POs and SDGs

6.1 Program Outcomes (POs) Mapping

Table 1: Mapping of Program Outcomes (POs) to Project Relevance

PO No.	Program Outcome	Relevance
PO1	Engineering Knowledge	Applies Python programming, strings, and modular arithmetic to implement six classical ciphers..
PO2	Problem Analysis	Breaks encryption and decryption into key validation, character processing, transformation, and output steps.
PO3	Design/Development of Solutions	Develops a Tkinter application with encryption, decryption, comparison, and analysis features.
PO4	Conduct Investigations of Complex Problems	Generates possible decryptions using key searches and candidate keywords for users to examine.
PO5	Engineering Tool Usage	Uses Python and Tkinter to implement and run a graphical application.
PO8	Individual and Collaborative Team work	Supports team collaboration in planning, implementation, documentation, and presentation.
PO9	Communication	Presents cipher results, comparison data, and step-by-step explanations through the interface and report.
PO11	Life-Long Learning	Provides a foundation for further study of programming, classical cryptography, and cryptanalysis.

6.2 Sustainable Development Goals (SDGs) Mapping

Table 2: Mapping of Sustainable Development Goals (SDGs) to Project Relevance

SDG No.	Goal	Relevance
SDG 4	Quality Education	Provides an interactive way to learn Python programming and introductory encryption concepts.
SDG 9	Industry, Innovation and infrastructure	Demonstrates the development of a software application that processes text and compares different algorithms.

6.3 Program Specific Outcomes (PSOs) Mapping

PSO No.	Program Specific Outcome	Relevance
PSO1	Apply software design and development practices in emerging areas such as Cloud, High-performance Computing, Data Analytics, and Cyber Security.	practices in emerging areas such as Cloud, High-performance Computing, Data Analytics, and Cyber Security.The project applies software development practices to create a Python application for demonstrating classical encryption and decryption methods. It introduces basic concepts related to cryptography and cyber security for educational purposes.
PSO2	Employ modern computer languages, environments, and platforms for entrepreneurship, innovative careers, and higher studies.	The project uses Python and the Tkinter interface to develop an interactive application. It builds programming and software development experience that can support further studies and future projects.

7 Conclusion

The CipherLab project developed a Python application with a Tkinter interface for demonstrating six classical encryption and decryption methods: Caesar, Atbash, Vigenère, Affine, Rail Fence, and Columnar Transposition. Users can select a method, enter text and the required key, and view the transformed result along with an explanation of the steps. The comparison feature applies all six methods to the same input and displays their outputs side by side. The analysis feature generates possible decryptions by trying key options or user-supplied keywords, helping users explore how classical ciphers can be examined.

The project applies Python concepts such as functions, loops, conditional statements, string handling, input validation, and event-driven programming with Tkinter. It also illustrates the difference between substitution ciphers, which replace letters, and transposition ciphers, which rearrange characters. CipherLab meets its educational goals by combining these algorithms with interactive explanations and comparison tools. However, the methods are classical ciphers with known weaknesses, and the analysis feature produces possible candidates rather than guaranteed solutions. The application is intended for learning and should not be used to protect confidential information.

Bibliography

Online Resources

[1] Python Software Foundation – *Tkinter: Python Interface to Tcl/Tk* https://docs.python.org/3/library/tkinter.html

[2] Python Software Foundation – *Built-in Functions* https://docs.python.org/3/library/functions.html

[3] Python Software Foundation – *Math: Mathematical Functions* https://docs.python.org/3/library/math.html

Research article references

[1] Al-Sabaawi, A. (2020). "Cryptanalysis of Vigenère Cipher: Method Implementation." *2020 IEEE Asia-Pacific Conference on Computer Science and Data Engineering (CSDE).* https://doi.org/10.1109/CSDE50874.2020.9411383

[2] Hananto, A. L., Solehudin, A., Irawan, A. S. Y., and Priyatna, B. (2019). "Analyzing the Kasiski Method Against Vigenere Cipher." *arXiv preprint arXiv:1912.04519.* https://arxiv.org/abs/1912.04519

[3] *Bruen, A., Forcinito, M., and McQuillan, J. (2021). "Classical Ciphers and Their Cryptanalysis." In Cryptography, Information Theory, and Error-Correction. https://doi.org/10.1002/9781119582397.ch2*

\